IERG3080 Tutorial 8

Crash Course on OO

Hoover H.F. Yin

Department of Information Engineering
The Chinese University of Hong Kong

November 2, 2017

Contact

- YIN Ho Fai, Hoover
- Office: Institute of Network Coding (INC), 7/F, SHB
- Email: yhf015 at ie dot cuhk dot edu dot hk

- Your local tutor
- Can speak Cantonese :D

Outline

OO

Object

Class

Inheritance

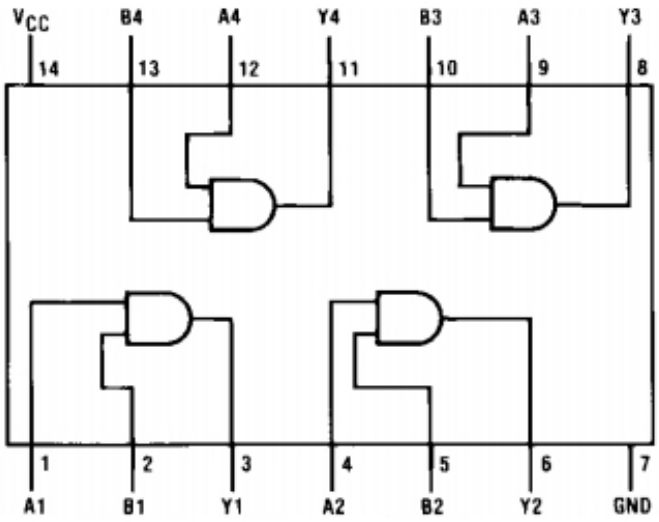
Misc.

Software Chip

Have you used those “hardware” chips before?

Source: http://www.electroschematics.com/8891/7408-datasheet/

Software Chip



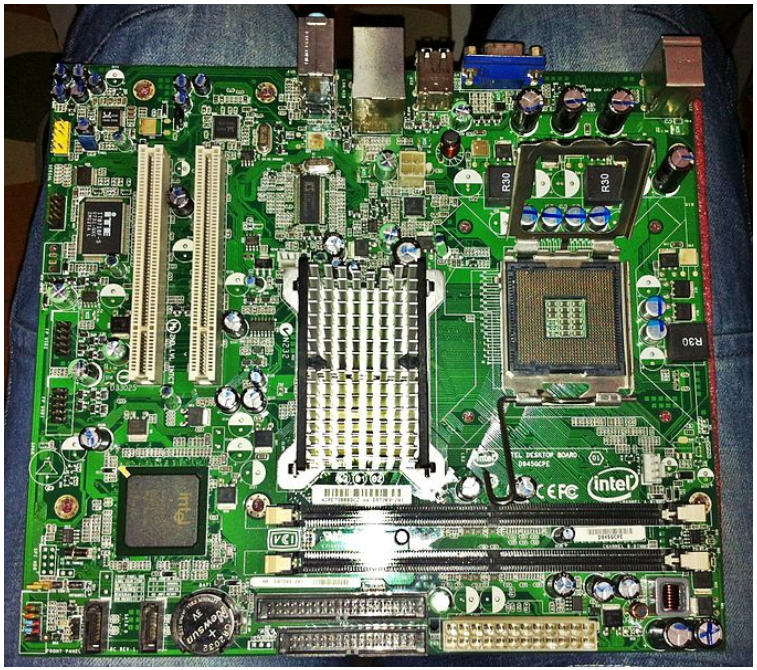
You only need to know

- What the chip can do
- How to use the chip

in order to use it. You do not need to know how it works

Source: http://www.electroschematics.com/8891/7408-datasheet/

Software Chip



You can have multiple instances of the same chip.
When you combine different chips (and other components), you can create a complex product

Source: https://en.wikipedia.org/wiki/Motherboard

Software Chip

Use this idea in software!

- You know what it can do
- You don't need to know why it works
- You can use it given the interface (pins assignment)
- Reusable – multiple instances

In a big project...

- Division of labour – everyone codes different chips
- But before you code, you need a design, e.g., how to use your chips

Problem Solving

Problem solving using a computer, you can design a solution which

- close to the machine
- close to the problem

Close to the machine?

- Design according to what the machine can do, e.g., C
- Run efficiently
- Machine friendly
- Good for small problem

Close to the problem?

- Design in the view of solving the problem, e.g., OO
- Not the most efficient way for the machine
- Human friendly
- Good for large problem

Object?

In real life, what is an object?

- Chair
- Car
- Sun
- Sand

How do you describe an object in real life?

Object?

How do you describe an object in real life?

- Method – What can it do?
  - Scissors – cut things
- Property – the features
  - My features – height: 1.x meter, weight: xyz kg
- Class – the model
  - e.g., iPhone X – same model but stored different data
- Inheritance – the same features for some classes
  - Humans and Monkeys – looks similar, intelligence
- Encapsulation – you can use it without knowing why it works
  - You don't know how the engine of a car works, but you can still drive
  - As a driver, you won't touch the internal part of the engine

Object!

Example: My iPhone 6 Plus

- Manufacturer – Apple
- Model – iPhone 6 Plus
- Color – Space Grey
- Storage – 128GB

What it can do?

- Make a phone call
- Install Apps
- Apple Pay

Object!

In the syntax of OOP...

```
//properties
myPhone.manufacturer = "Apple";
myPhone.model = "iPhone 6 Plus";
myPhone.color = "Space Grey";
myPhone.storage = 128;

//methods
myPhone.Call();
myPhone.InstallApps();
myPhone.ApplePay();
```

Class vs. Object

Something wrong?

- Human skipped the lecture.

This seems make sense

- William skipped the lecture.

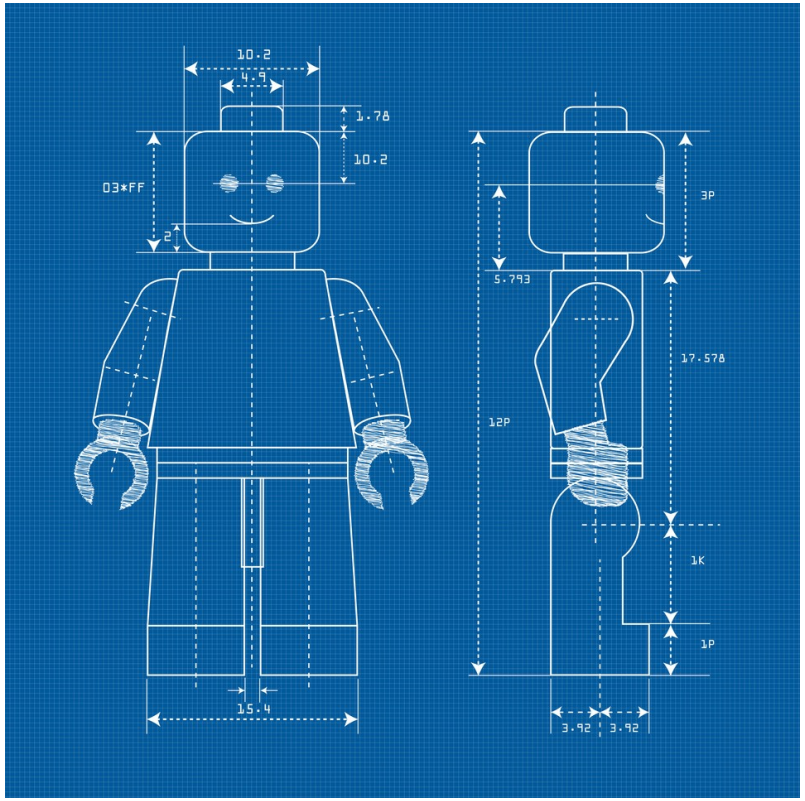
So what's wrong with the first sentence?

- William is a human
- "Human" does not correspond to a specific individual

Human is a class, William is an object

- Class is a blueprint
- Object is an individual

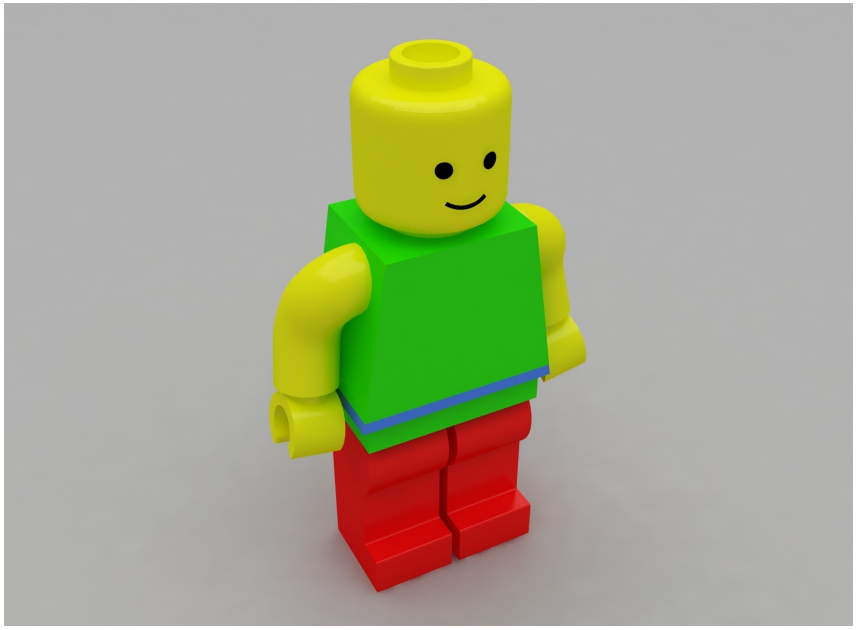
Class vs. Object



This blueprint specifies how a Lego human looks like, but you don't have a Lego human on hand!
This IS a class.

Source: https://crated.com/art/141810/lego-blueprint-by-alessandroaru

Class vs. Object



According to the blueprint, you can make a Lego human, as much as you want.
This IS an object. You can have multiple objects according to the same class.

Source: http://www.turbosquid.com/3d-models/lego-human-3d-model/711309

Terminology

In a nutshell...

- Field – The variables in the object
- Property – Act like variables but having getter, setter
- Method – Functions, e.g., `myPhone.Call()`
- Modifier – Access permission, e.g., `public`, `private`, `protected`

Getter/Setter

Make everything under your control... Users are evil!!
I won't allow someone to state my iPhone model to be Note 7!!

Well??? What does it mean?

```
public int favNum { get; set; }
```

- Auto property – the compiler creates the field for you
- Easy to read – it's a property for sure
- Easy to maintain – what if you change the spec?
  - It is a trouble to covert a field in to a property!
    - Although the syntax are the same, but accessing a field is accessing a variable, while using getter/setter is calling functions!
    - You need you compile other codes again after the change although the codes are the same

Create an object

```
Random rand = new Random();
Random rand2 = new Random(1999);
```

When you `new` an object... The process does the following

- Allocate a memory space for the object
- Initialize the fields
- Run the constructor

Which constructor? It is specified after your `new` operator.

Constructor

```
public class MyClass
{
    public MyClass(){} // the default constructor
}
```

- The compiler will make a default constructor for you if...
  - you have not coded any custom constructor
- Constructor is not a method, although it looks like a function
- No return type indicated
- The same name as the class name

Is it always `public`?

- If you don't want the object to be created, you can use `private`, e.g., the `Math` class

Inheritance

What a smart phone can do?

- Make a phone call
- Install Apps

```
public class SmartPhone
{
    public bool Call(string no){}
    public bool InstallApps(int appID){}
}
```

Different smart phone has different extra functions

Btw do you know why I use `string` to represent a phone number?

- What if the number is too long? (`int` overflow)
- What if the number contains *, # or +?

Inheritance

So we use inheritance to indicate the extra functions

```
public class iPhone : SmartPhone
{
    public bool ApplePay(){}
}

public class Note7 : SmartPhone
{
    public void Explode(){}
}
```

In C#, before running its own constructor, it will first run the constructor of its parent class.

Dynamic Polymorphism

We can call an iPhone or a Note 7 a SmartPhone, but we can't call a SmartPhone a Note 7 unless it's really a Note 7. However, we call make a phone call without knowing it is iPhone or Note7.

```
iPhone myPhone = new iPhone();
SmartPhone mySmartPhone = myPhone; //OK
iPhone myPhone2 = (iPhone) mySmartPhone; //OK
Note7 myPhone3 = (Note7) mySmartPhone; //NOT OK

mySmartPhone.Call("21800000"); //OK
```

- The cast is done in run time (so it throws an exception)
- It searches the method `Call()` in run time too
  - It will call the corresponding method if you override it (next slide)
- This is a kind of dynamic polymorphism

Override

How about there's a default behaviour? Only some of the derived classes have different behaviours? Mark it `virtual` then `override` it!

```
public class Human
{
    virtual public void Sleep(){
        Console.WriteLine("In the bedroom");
    }
}

public class Student : Human
{
    override public void Sleep(){
        Console.WriteLine("In the classroom");
    }
}
```

Abstract Class

```
public class Animal
{
    public void Talk(){}
}
```

Different `Animal` talks in different way...

- We have a default behavior if we use `virtual`
- How to make sure the derived classes implement this method?

Abstract Class

```
abstract public class Animal
{
    abstract public void Talk();
}

public class Cat : Animal
{
    override public void Talk(){
        Console.WriteLine("Meow");
    }
}
```

- Note that you can't `new` an abstract class
- An abstract method cannot be `private` as it is meant to be overridden.
- However, it can be `protected`

Interface

Interface is a contract. Once you promise to implement the interface, you MUST provide the services (implement the methods). As it tells the others how to use your class, of course the methods in the interface are `public`.

- `Rectangle` – `width`, `height`
- `Circle` – `radius`
- Not so alike but both should provide the method `Area()`

```
public interface Shape
{
    double Area();
}
```

- You don't have to write `public` in the interface declaration (while abstract method can be `protected`)

Interface

```
public class Circle : Shape
{
    public double radius {get;set;}
    public double Area(){return radius*radius*Math.PI;}
}

public class Rectangle : Shape
{
    public double width {get;set;}
    public double height {get;set;}
    public double Area(){return width*height;}
}
```

static

- `Math.Sqrt()`
  - Hey, `Math` is a class
  - Do you need to create an object?

This is how this method declared:

- `public static extern double Sqrt(double d);`

This is a class method. You don't need to create an object before you use it!

Generic Programming

Suppose I have the following swap function for `int`

```
public static void swap(ref int a, ref int b)
{
    int tmp;
    tmp = a;
    a = b;
    b = tmp;
}
```

If I want to have a swap function for `double`, why I need to copy the whole method but only changing the data types???

Generic Programming

Remember that < >? You used that before! (e.g., `List<int>`)

```
public static void swap<T>(ref T a, ref T b)
{
    T tmp;
    tmp = a;
    a = b;
    b = tmp;
}
```

We can then use `swap<int>(a,b)` and `swap<double>(a,b)` and you don't need to copy the codes :D

- The compiler creates both methods for you, at compile time
- This is a kind of static polymorphism (as it's not done at run time)

Method Overloading

You can create methods with the same name but different signature

- Signature includes the method name and its parameters' types (NOT names), but NOT the return type

```
public void Hi(int a){}
public void Hi(double a){} //OK
public int Hi(int a){} //NOT OK
public int Hi(int b){} //NOT OK
```

Operator Overloading

You can create you own definition of the operators

- e.g., why we can use + in `string`

```
public class Test
{
    public int a;
    public Test(int a){
        this.a = a;
    }
    public static Test operator+(Test s, Test t){
        return new Test(s.a + t.a);
    }
}
```

- Now suppose we have two `Test` objects called `foo` and `bar`
- We can now perform `foo + bar`

Summary

Object-Oriented Programming

- Making modules which you can use them without knowing how they work
- Easy to divide the jobs in a large project

Class

- A blueprint which indicates how an object works
- It has no instance

Object

- A instance created according to the class
- We can create multiple instances of the same class and store different data

Field

- The variables in the object
- They are really variables

Summary

Property

- The syntax to use a property is the same as using a field
- They are not variables, but composite of two functions: getter and setter

Method

- The functions in the object
- It can change the data of the object so you have to create an object first (unless it is a class method)

Class Method

- A class method is defined with the keyword `static`
- It does not need an object to work on

Modifier

- `public`: everyone can access it
- `private`: can only access it inside the object
- `protected`: only the object / its derived objects can access it

Summary

Constructor

- When creating an object, the constructor will be run
- If no custom constructor is defined, the compiler will create the default constructor for you
- It is not a method, although it looks like a function
- No return type indicated, and it must have the same name as the class name
- Usually it is `public`

Inheritance

- The derived class gains access to all `public` and `protected` fields, properties and methods of the base class
- The constructor of its base class will be run before its own constructor

`virtual`

- A keyword which indicates the method can be overridden

Summary

Abstract Class

- A class with a method marked with `abstract`
- No implementation is provided for the abstract methods
- No object of this class can be created
- These abstract methods are meant to be overridden, so they cannot be `private`

Interface

- A promise to implement the methods
- Other classes can use these methods as you must implement them, and so they must be `public` during implementation

Generic Programming

- Coding with a generic type
- The code for the specific types are created by the compiler

Summary

Method Overloading

- Methods of the same name but different signature (method name + parameters' types)
- The parameters' names and the return type are not a part of the signature

Operator Overloading

- A class method to indicate how an operator works for the class